



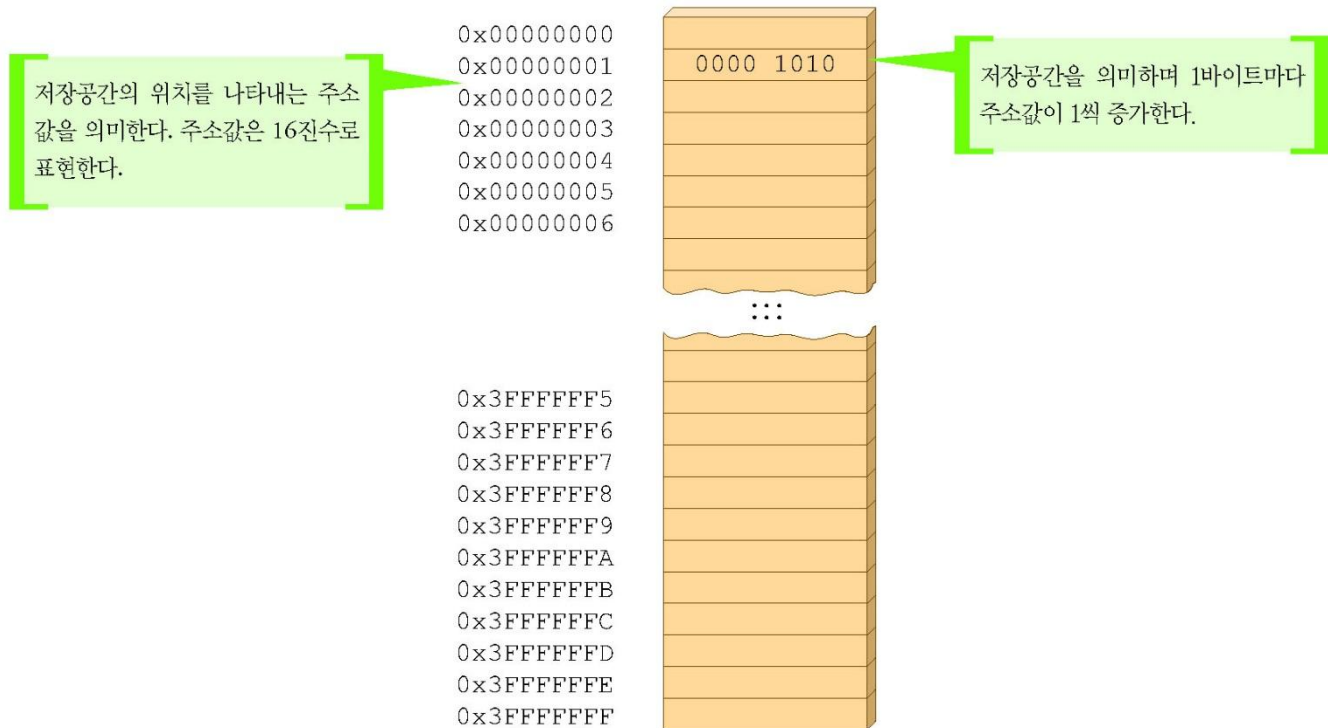
Computer Programming I (CSE2003)

9. Pointer

Memory Address

□ Address(주소)

- 메모리에는 그 메모리의 저장 장소의 위치를 나타내는 주소 값
- 주소(address)는 1바이트마다 1씩 증가하도록 메모리에는 연속적인 번호가 구성



주소 연산자

□ &변수

- 변수의 주소 값을 알아내려면 변수 앞에 주소 연산자 & (ampersand)를 이용

□ 주소 값 이용 장단점

- 주소 값을 이용하면 보다 편리하고 융통성 있는 프로그램이 가능
- 그러나 복잡하고 어려워서 코딩 에러가 잘 나는 단점

```
int age;  
printf("%p, %u", &age, &age);
```

변수 age 앞에 &를 기술하면 age의 주소값이 반환되고, 이를 출력하려면 변환명세 %p를 이용한다.



Concepts of pointers

□ Pointer values

```
// Print character addresses  
#include <stdio.h>
```

```
int main (void)
```

```
{  
// Local Declarations
```

```
char a;
```

```
char b;
```

```
// Statements
```

```
printf ("%p\n %p\n", &a, &b);
```

```
return 0;
```

```
} // main
```

주소 연산자를 이용한
변수 a, b의 출력

변수 a, b로 할당된
메모리의 주소

a

142300

b

142301

142300

142301

- 컴퓨터는 char 변수 a, b가 선언될 때 저장공간을 1byte씩 제공
- 위 프로그램의 결과로 변수 a, b의 주소 값 142300, 142301이 출력

포인터 변수

- 포인터 변수는 일반 변수와는 다르게 변수에 저장되는 값이 메모리의 주소 (address) 값만을 저장할 수 있는 특별한 변수
- 포인터 변수를 이용하면 프로그램이 간결하고 효율적이므로 포인터 변수를 이용

변수 :



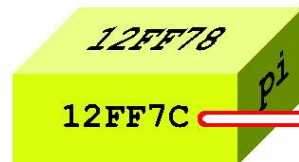
하나의 변수는 변수명, 변수에 저장된 저장값, 그리고 저장공간의 주소 값을 갖는다.

일반 변수 :



```
int i = 3;
```

포인터 변수 :



```
int *pi = &i;
```

- 변수 이름은 그 변수가 선언된 함수에서만 볼 수 있지만, 그 변수의 주소를 알면 모든 함수에서 접근 가능

- 포인터 변수는 주소를 값으로 갖는 변수.

포인터 변수 선언

□ 포인터 변수의 선언

- 포인터 변수는 그 포인터가 가리키는 변수의 자료형에 맞추어 형을 선언해야 함
- 포인터 변수임을 나타내 주는 *
- 별표 *의 위치가 변수 자료형 int와 변수명 사이 어디에 위치하든 관계 없음

int는 포인터 변수 ptr에 저장되는 주소의 변수가 갖는 자료형을 나타낸다.

변수 ptr은 자료형 int의 포인터라고 말하며, *ptr 자체가 자료형 int의 변수라고 생각할 수 있다.

int *ptr;

□ 여러 개의 포인터 변수를 한 번에 선언

- 다음은 ptr1만 포인터 변수

int* ptr1, ptr2, ptr3;

- 세 변수를 모두 포인터 변수로 선언하려면 다음과 같이

int *ptr1, *ptr2, *ptr3;

포인터 변수의 값

□ 포인터 변수는 주소(address) 값만을 저장

- 포인터 변수도 초기 값 지정 가능

```
int i;           //double phi = 3.14;  
int *ptr = &i;   //double *ptr = &phi;
```

- 위 구문은 변수 ptr에는 변수 i의 주소 값이 저장
- 변수 ptr은 초기 값이 변수 i의 주소가 지정되었으므로 *ptr은 변수 i 자체를 의미

```
int i = 3;  
int *ptr = &i;
```



역참조 연산자(Indirection Operator)

- 역참조 연산자(Indirection operator) *
 - 역참조 연산자는 포인터 변수 앞에 연산자 *를 붙이면 그 포인터가 가리키는 변수를 지칭
- 구문 *ptr = i + 2;을 이용 (i = i + 2; 와 같은 명령)
 - 변수 i의 값이 2 증가
 - 연산자 *는 포인터 변수를 뒤에 기술하면 역참조(dereference) 연산자

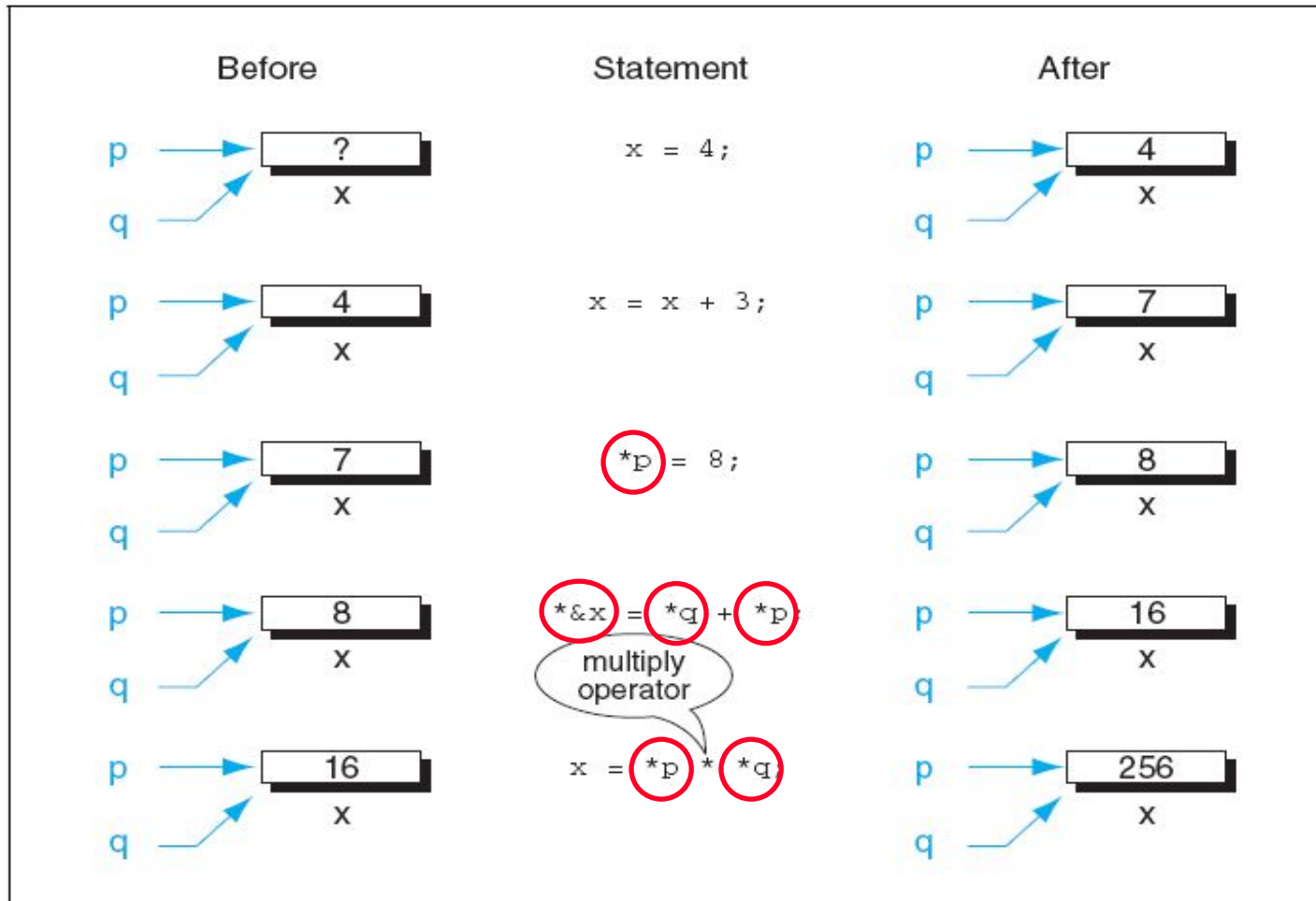
```
int i = 3;  
int *ptr = &i;
```

```
*ptr = i + 2;
```

*ptr은 ptr이 가리키는 변수 자체를 의미한다. 여기서는 ptr이 변수 i의 주소값을 가지므로 *ptr은 변수 i를 의미한다.

Accessing variables through pointers

- p, q가 변수 x를 가리키는 포인터 변수일 때, 다음과 같이 변수 x를 access하는 여러 가지 방법이 있다.



○ 표 한
것은 모두 'x'
에 해당한다.

Pointer declaration and definition

□ Pointer variable declaration

data declaration

type

identifier

pointer declaration

type *

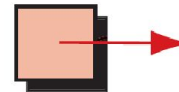
identifier

□ 서로 다른 타입을 참조하는 포인터 변수의 선언

`char a;`

z

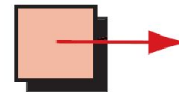
`char* p;`



`int n;`

15

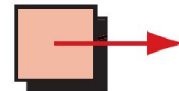
`int* q;`



`float x;`

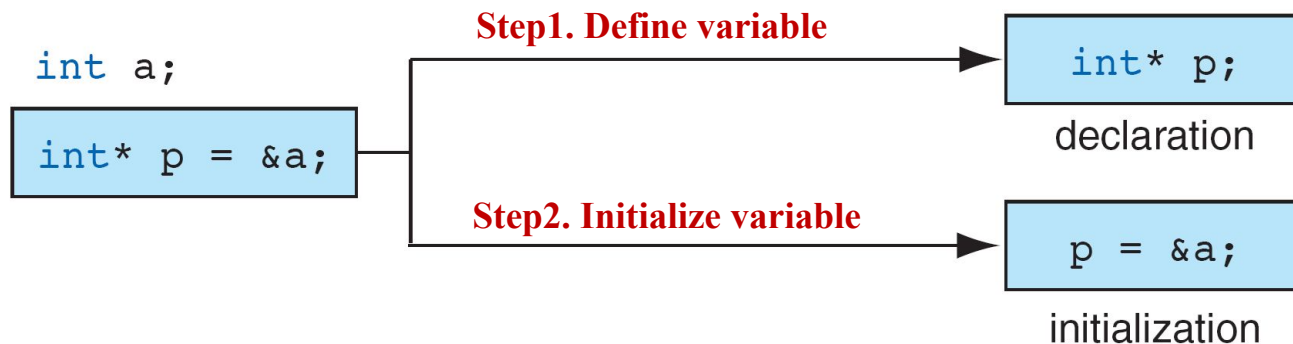
3.3

`float* r;`



Initialization of pointer variables

- 일반적인 변수가 선언된 후에 초기화되지 않으면 쓰레기 값을 갖는 것처럼 포인터 변수도 초기화하지 않으면 엉뚱한 곳을 가리키게 된다.
- 포인터 변수는 주소값을 가지므로 주소 연산자(&)를 이용하여 초기화한다.



- 포인터 변수가 아무 변수도 가리키지 않도록 하려면 다음과 같이 초기화한다. `*p = NULL;`

포인터 사용의 예

선언 및 초기화

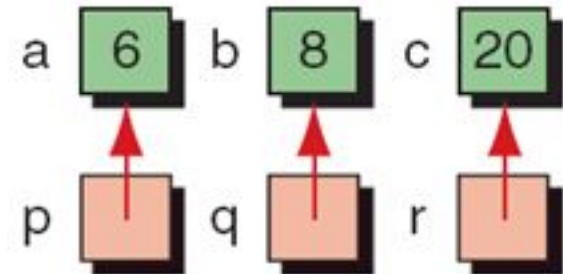
```
int i = 3, j = 5, *p = &i, *q = &j, *r;  
double x;
```

수 식	등가 수식	값
<code>p == & i;</code>	<code>p == (& i);</code>	1
<code>* * & p;</code>	<code>* (* (& p));</code>	3
<code>r = & x;</code>	<code>r = (& x);</code>	/* illegal */
<code>7 * * p / * q + 7;</code>	<code>((7 * (* p)) / (* q)) + 7;</code>	11
<code>* (r = & j) *= *p;</code>	<code>(* (r = (& j))) *= (* p);</code>	15

Example program

예제 프로그램 - 포인터 변수를 사용한 예제

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a, b, c;
6     int *p, *q, *r;
7
8     a = 6;
9     b = 2;
10    p = &b;
11
12    q = p;
13    r = &c;
14
15    p = &a;
16    *q = 8;
17
18    *r = *p
19
20    *r = a + *q + *&c;
21
22    printf("%d %d %d\n", a, b, c);
23    printf("%d %d %d\n", *p, *q, *r);
24
25    return 0;
26 }
```



```
[root@mclab chap9]# vi chap9-2.c
[root@mclab chap9]# gcc -o chap9-2 chap9-2.c
[root@mclab chap9]# ./chap9-2
6 8 20
6 8 20
[root@mclab chap9]#
```

Pointers and functions

▣ 참조에 의한 호출(Call-by-reference)

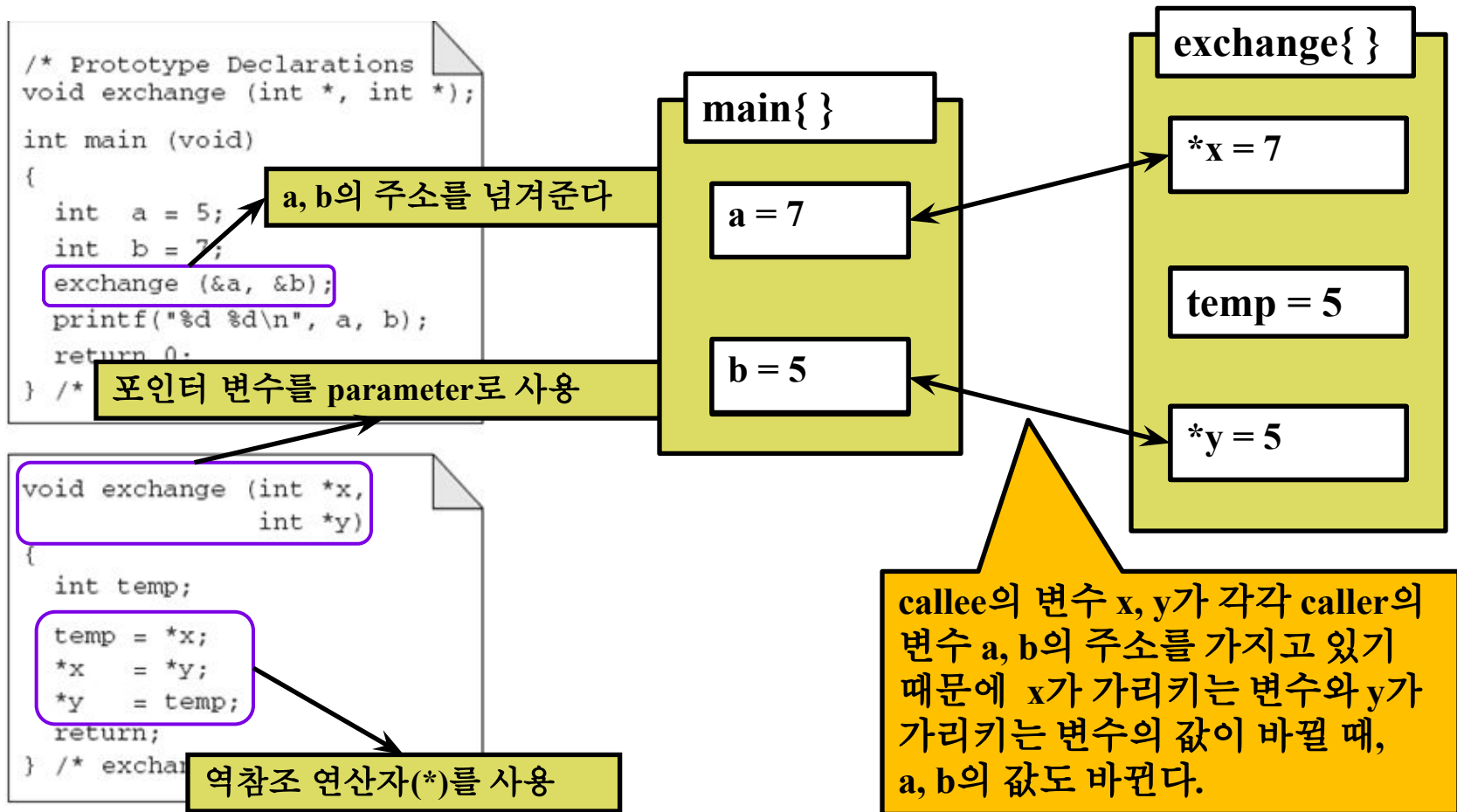
- 참조에 의한 호출은 caller에서 callee에 parameter를 넘길 때
 - 변수의 값을 넘겨주는 대신 변수의 주소를 넘겨주는 방식
- callee에서 caller의 변수에 대한 주소를 가지고 있기 때문에 callee에서 caller의 변수의 값을 변경할 수 있다.
- 참조에 의한 호출을 사용하면 앞서와 같은 문제를 피할 수 있다.

▣ "참조에 의한 호출"의 효과를 얻는 방법

- 함수(callee)의 parameter를 포인터형으로 선언
- Caller에서 함수(callee)를 호출할 때 parameter로 주소를 전달
- 함수(callee) 내부에서 parameter 사용시 역참조 연산자(*) 사용
 - 넘겨받은 값이 주소값이므로 그 값을 사용할 때는 당연히 역참조 연산자를 사용해야 한다.

Pointers and functions

- Call by reference 방법을 이용하여 해결한 방법



Pointers and functions

▣ Functions returning pointer

- 다음 프로그램과 같이 함수가 pointer 변수를 리턴할 수도 있다.
- 이 경우에는 함수의 헤더 부분에서 return type을 pointer형으로 해주어야 한다.
- 지역 변수에 대한 참조를 리턴하는 것은 심각한 오류를 불러올 수 있다.

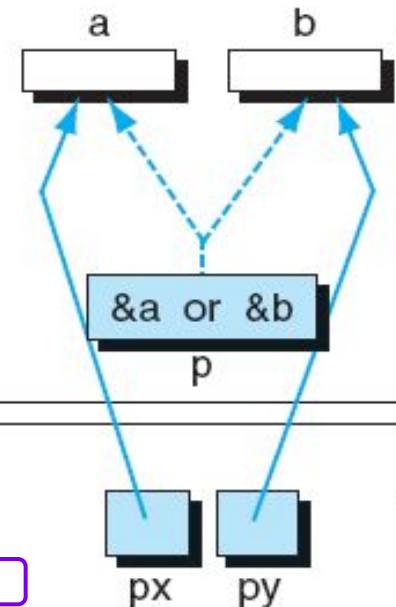
Return type이 pointer형이다.

Pointer type을 return한다.
(조건식의 결과에 따라 작은 값을 가진 변수에 대한 참조를 return한다.)

```
/* Prototype Declarations */
int *smaller (int *p1, int *p2);

int main (void)
...
int    a;
int    b;
int    *p;
...
scanf ( "%d %d", &a, &b );
p = smaller (&a, &b);
...
```

```
int *smaller (int *px,
              int *py)
{
    return (*px < *py ? px : py);
} /* smaller */
```



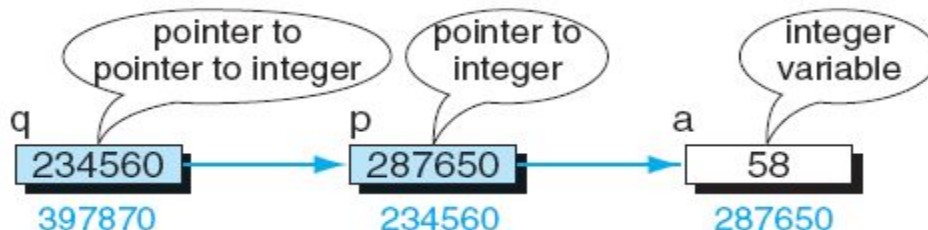
Pointers to pointer

- 포인터 변수가 다른 포인터 변수를 참조할 수도 있다.
 - 다음 프로그램은 int형 변수 a를 참조하는 포인터 변수 p와, 포인터변수 p를 참조하는 포인터변수 q에 대한 예제이다.

Pointer변수를 참조하는
pointer 변수의 선언

p는 a를, q는 p를
참조한다.

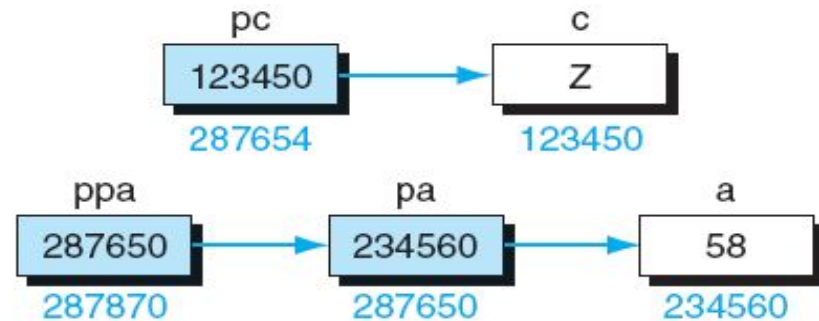
```
/* Local Declarations */  
int    a;  
int    *p;  
int    **q;  
  
/* Statements */  
a = 58;  
p = &a;  
q = &p;  
  
printf(" %3d",  a);  
printf(" %3d",  *p);  
printf(" %3d",  **q);
```



- integer형 포인터 변수를 참조하는 포인터 변수 q의 선언 방법은 다음과 같다.
 - int **q;
- q는 p를 통해 a를 참조할 수 있다.
- q를 이용하여 a를 참조하기 위해서는 두 단계를 거쳐야 하므로 **q와 같이 역참조 연산자를 두 번 사용한다.
- 따라서 a, *p, **q에 의해 출력되는 결과는 모두 58이다.

Compatibility

- 포인터 변수에 다른 타입의 주소를 저장하면 error가 발생한다.



```
char c;  
char *pc;  
  
int a;  
int *pa;  
int **ppa;  
  
pc = &c;           /* Good and valid */  
pa = &a;           /* Good and valid */  
ppa = &pa;        /* Good and valid */  
  
/* Invalid pointers will generate errors */  
pc = &a;           /* Different types */  
ppa = &a;          /* Different levels */
```

Char 타입 포인터에 int형 변수의 주소를 저장 → error!

int * 타입 포인터에 int형 변수의 주소를 저장 → error!

Compatibility

□ Void pointer

- 포인터 변수도 type을 가지고 있기 때문에 다른 type의 변수를 참조하도록 한다면 compile error가 발생한다.
- Void pointer는 임의의 type을 갖지 않기 때문에 어떤 type이든 참조할 수 있다.
- 선언 방법 : `void *pVoid;`

□ Casting pointer

- 선언된 포인터 변수가 다른 타입의 변수를 참조할 수 있도록 강제적인 형 변환이 가능하다.
- Ex) `int a;`
`char *p;`
`p = (char *)&a;`
- 이러한 형 변환은 메모리의 낭비를 불러올 수 있다.

Compatibility

▣ Casting pointer

- 다음은 void pointer를 이용한 변수의 참조와 casting을 통한 참조의 예제이다.

```
/* Local Definitions */
void *pVoid;
char *pChar;
int *pInt;
/* Statements */
pVoid = pChar;
pInt = pVoid;
pInt = (int *) pChar;
```

- 둘 다 pChar을 pInt에 저장하고 있다.
 - 첫 번째 경우는 다른 type의 pointer 변수라도 참조가 가능한 void 타입 포인터를 이용했다.
 - 두 번째 경우는 type casting을 이용했다.

실습 9-1

실습 9-2